

ORCS 6529 · ADVANCED REINFORCEMENT LEARNING · FINAL
PRESENTATION

Dynamic Resource Allocation in Cloud Computing

*A power-aware, SLA-bound deep-RL scheduler — built end-to-end
from Gymnasium simulator to FastAPI dashboard, ~5.2K LOC*

Tanish Patel · tp2899

Arnav Ahuja · aa5790

Why power-aware scheduling, and why now?



THE PROBLEM

~1–2%

of global electricity is consumed by data centers (IEA, 2024) — and the share is rising fast with AI workloads.

Server power is non-linear: $P(u) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \cdot u^\alpha$

Goal

Build a real-time scheduler that decides which queued job runs on which server — minimising cluster power while honouring SLA latency deadlines, and generalising across unseen workloads.

Prior approaches



Static heuristics

Round Robin, Shortest-Job-First, First-Fit Decreasing — fast and stateless, but blind to power non-linearity, server tiers, and bursty arrivals.



Control / ILP / bin-packing

MIP and online bin-packing — strong guarantees on small instances, but scale poorly and break under workload uncertainty.



Supervised learning

Optimal scheduling on a 50-server cluster is NP-hard, so label generation is intractable; supervised models also cannot capture long-horizon power × SLA trade-offs.

Why reinforcement learning is the right fit

How others have framed it

2016 DeepRM (Mao et al.)

First end-to-end RL scheduler — REINFORCE policy gradient over a multi-resource cluster simulator.

2019 Decima · RL-VM placement

Graph-NN-based RL for DAG-structured Spark jobs and VM consolidation in cloud DCs.

2019+ Energy-aware RL schedulers

DQN / A3C variants minimising $PUE \times SLA$, mostly on single-DC simulators with hand-tuned weights.

2021+ Decision Transformers & CMDPs

Sequence-modelling and constrained-MDP formulations bring formal guarantees and offline-data learning.



Why RL (and not heuristics or SL)?



Sequential, delayed reward

Daily energy & SLA outcomes depend on long sequences of placement decisions — exactly what γ -discounted return optimises.



Adapts to randomized and bursty arrivals

Trial-and-error policies adjust online to workload regimes that fixed heuristics cannot see.



Captures non-linear power

Learns u^α curves directly from reward — no hand-tuned utilisation thresholds or per-tier constants.



Constraint-aware variants

CMDP / Lagrangian formulations let SLA become a hard expected-cost budget, not a fragile penalty term.

Modelling the cluster as a discrete-time MDP

$M = \langle S, A, P, r, \gamma=0.99 \rangle$ · custom Gymnasium env · BASELINE: $N = 50$ servers (all-on), $K = 5$ queue slots, 1000-step episodes



STATE S_t ($4 \cdot N + 4 \cdot K$ dim)

- Per server ($\times N$): cpu_util, mem_util, p_max_norm, cpu_cap_norm
- Per queued job ($\times K$): cpu_req, mem_req, duration, wait_time
- Tier identity exposed so the agent can prefer efficient servers



ACTION A_t · Discrete($K \cdot N + 1$)

- (k, n) jointly: WHICH queued job $\rightarrow$ WHICH server (251 choices)
- “Wait” action — gated to legal only when nothing currently fits
- Invalid-action mask applied at selection AND at TD bootstrap



DYNAMICS P · all servers always-on

- 5 workload generators: synthetic Poisson, Alibaba '18, Google v2/v3, sampled-hybrid
- Servers run jobs to completion; queued jobs may expire (SLA breach $\rightarrow$ dropped)
- Heterogeneous fleet — efficient · standard · power-hungry tiers



REWARD r_t (heart of the env)

- $r = -(\alpha \cdot P_{\text{active}} + \beta \cdot \text{SLA_signal})$ $\alpha = 1, \beta = 50$ (baseline)
- $P_{\text{active}} = (P_{\text{total}} - \sum P_{\text{idle}}) / \sum (P_{\text{max}} - P_{\text{idle}}) \in [0,1]$
- SLA_signal = discrete violations + continuous queue-pressure

Four learned agents — three RL paradigms

VALUE-BASED

Double DQN

Discrete · off-policy

Why this paradigm

Natural fit for a discrete $K \cdot N + 1$ action space; replay buffer reuses every transition. Masking applied in BOTH selection AND the TD bootstrap target — without the second step, illegal actions silently bleed into Q_{target} and the policy never converges.

Key choices

- ϵ -greedy $1.0 \rightarrow 0.05$ (exp decay 0.999)
- Target net hard-update / 1k steps
- $256 \rightarrow 256$ MLP, ReLU, smooth-L1, Adam $3e-4$

POLICY-GRADIENT

PPO (Actor-Critic)

On-policy · clipped surrogate

Why this paradigm

Stable on-policy actor-critic — lets us study whether stochastic policies handle bursty arrivals better than a greedy Q -policy. Uses a custom outer loop because PPO can't reuse stale data. Strong empirical baseline for fair RL-vs-RL comparison.

Key choices

- $GAE\text{-}\lambda = 0.95$, clip = 0.2, 10 epochs
- Masked Categorical with -inf logits
- Adv. whitening + $\text{std} < 1e-6$ guard, grad clip 0.5

AGENTIC RL

Supervisor + Sub-agents

Hierarchical · multi-objective

Why this paradigm

Replaces the brittle (α, β) reward trade-off with a learned arbitrator. Two sub-agents (power, SLA) propose independently; a supervisor draws a Bernoulli over their proposals, trained with REINFORCE on the combined cluster reward.

Key choices

- SLA-Agent (DQN, reward = -viol.)
- Power-Agent (DQN, reward = -P)
- Supervisor MLP $[3 \rightarrow 32 \rightarrow 32 \rightarrow 1]$, REINFORCE

OFFLINE / SAFE

Lagrangian CMDP

Offline FQI · constraint-aware

Why this paradigm

Treats SLA as a hard expected-cost budget rather than a soft penalty term — formal constraint guarantee with no live exploration risk. Trains on a logged-transitions dataset (FQI), making it the only agent that can be deployed without online interaction.

Key choices

- Q_r, Q_c via FQI / Double-DQN targets
- $\pi = \text{argmax}(Q_r - \lambda \cdot Q_c)$
- $\lambda \uparrow$ on violation, $\lambda \downarrow$ on slack (dual ascent)

Two design ideas that mattered most

Reward shaping

$$r_t = -(\alpha \cdot P_{\text{active}} + \beta \cdot \text{SLA_signal} + \gamma_t \cdot \text{Toggle})$$

- Active-power normalisation $P_{\text{active}} \in [0, 1]$**
 Subtract per-server idle floor and divide by the active ceiling — the agent only sees the slice of power it can actually move. Removes the constant-idle bias that otherwise dominates the gradient.
- Continuous queue-pressure shaping**
 Discrete violations fire only at deadline expiry. We add $\sum \max(0, -\text{headroom_norm})$ so a useful gradient exists throughout the buildup phase, where actions actually matter — not just at the cliff.
- Invalid-action penalty + double masking**
 -10 for impossible (job, server) pairs; mask applied at selection AND inside the TD bootstrap — closes the silent leak that lets bad actions contaminate Q_{target} .
- Coefficients $\alpha = 1, \beta = 50$ (baseline)**
 Power normalised in $[0, 1]$, SLA signal un-normalised $\rightarrow \beta$ must be $\gg \alpha$ to keep SLA the dominant short-term gradient. Re-tuned to $\alpha = 5, \beta = 60$ once sleep actions made power a controllable signal (see extension slide).

State encoding & network

$$\text{State vector} = 4 \cdot N + 4 \cdot K = 220\text{-dim (baseline)}$$

4·N server features

4·K queue features

Linear 256 $\rightarrow$ ReLU/Tanh $\rightarrow$ Linear 256 $\rightarrow$ Activation

Q-head / Policy head
 $\text{dim} = K \cdot N + 1$
 (masked with $-\infty$ on illegals)

Value / Cost head
 $V(s)$ or $Q_c(s, a)$
 (CMDP variant only)

Same trunk reused across DQN, PPO and CMDP — only the heads differ. Sleep-aware extension grows N -channel inputs to $6 \cdot N$ and the action head to $K \cdot N + N + 1$.

Designing for fair, reproducible comparisons



Train / test seed pools

ML-style split: 50 train + 50 test seeds, offset by 10^6 so workload streams cannot overlap. Greedy eval on the held-out pool — every reported number is on data the agent has never seen.



Action masking everywhere

Mask applied at selection AND inside the TD bootstrap (`Q_next.masked_fill(-inf)`). Skipping the second step is the silent DQN-killer — illegal actions become high-Q targets and the policy diverges.



Deterministic heterogeneous fleet

`cluster_seed = 0` fixed across every run → identical 3-tier (efficient/standard/power-hungry) fleet for every agent. Removes “fleet-luck” from A/B comparisons; all reward gaps are policy-driven.



Trace-sampled hybrid workload

Real Google-v2 (cpu, mem, duration) marginals + tunable Poisson timing — solves the pure-replay saturation that flattens every agent at ~99% SLA, while keeping realistic heavy-tail job specs.

EXPERIMENTATION HARNESS · ~5.2 K LOC end-to-end

Built so a teammate can replay any run from a single .env + Pydantic request body.



Gymnasium env
+ 5 workload gens

PyTorch agents



Generic Trainer +
Evaluator + WandB

logs/<run>.log



FastAPI orchestrator
+ ThreadPoolExecutor

POST /api/train



SQLite store +
WebSocket bus

live metrics



React dashboard
(Train · Monitor · Compare)

Recharts UI

Adding sleep / wake server actions

Servers can now be put to sleep (standby = 5 % · P_{idle}) — makes scheduling decisions move power, not just SLA.



STATE S_t · extended ($6 \cdot N + 4 \cdot K$)

- Two new per-server channels: $sleep_state \in \{asleep, waking, active\}$
- $wake_remaining$: countdown of unavailable steps after a wake action
- Same masking trunk reused — sleeping servers are illegal dispatch targets



DYNAMICS P · sleep / wake model

- Asleep: draws $SLEEP_STANDBY_FACTOR \cdot P_{idle} = 5\%$ of idle power
- Wake takes $WAKEUP_DELAY = 1$ step; server draws full P_{idle} , cannot accept jobs
- Cluster shrunk $N = 50 \rightarrow 15$ so dispatch + sleep choices actually matter



ACTION A_t · Discrete ($K \cdot N + N + 1$)

- Dispatch region ($K \cdot N$): same as baseline — assign queued job k to server n
- Toggle region (N): flip server n between active $\leftrightarrow$ asleep
- Wait (1): gated legal only when nothing fits and no useful toggle exists



REWARD r_t · sleep-aware shaping

- $r = -(5 \cdot P_{active} + 60 \cdot SLA_signal + TOGGLE_PENALTY \cdot 1[toggled])$
- Asymmetric toggle: 0.2 paid on sleep, free on wake — discourages flicker
- $\alpha 1 \rightarrow 5, \beta 50 \rightarrow 60$: power becomes a real signal once it is controllable

How the sleep / wake extension was built



Action-space surgery

ClusterEnv.action_space grows from $K \cdot N + 1$ to $K \cdot N + N + 1$. A small dispatcher routes each id to dispatch / toggle / wait, so the same agent code works for both the baseline and sleep-aware modes — switched by a single env flag.



Power-model rewrite

Per-server power query is now branch-free over {asleep, waking, active}. Asleep draws $SLEEP_STANDBY_FACTOR \cdot P_idle$; waking draws full P_idle but is masked from the dispatch action set until `wake_remaining = 0`.



Toggle-penalty design

Per-action cost charged only on sleep transitions, not on wakes. Asymmetry kills bang-bang flicker (which pays the wake-up tax without ever realising standby savings) yet leaves wake-up free when load arrives.



Capacity + reward re-tune

$N \ 50 \rightarrow 15$ makes scheduling matter (was $\sim 10\times$ over-provisioned). `SLA_DEADLINE` $10 \rightarrow 30$ lifts the workload-imposed SLA floor that pinned every agent at 12.9%. $\alpha \uparrow, \beta \uparrow$ rebalances the now-controllable power term.

BEFORE → AFTER · what changed end-to-end

Each box names one component of the stack and how it was modified to support sleep/wake.



ClusterEnv
+ sleep/wake actions

$K \cdot N + 1 \rightarrow K \cdot N + N + 1$



Server model
+ 3-state machine

active/wake/sleep



Reward shaping
+ toggle penalty

asym. 0.2 / 0.0



Config / .env
 $N=15$, deadline 30

$\alpha=5, \beta=60$



Frontend defaults
Train + Sweep pages

in-lockstep w/ env

Poisson workload · N=50 het. fleet · 500 episodes



Held-out test pool · 50 unseen seeds · greedy eval

Agent	Mean R	Mean Power	SLA rate
SJF	-91.2	1,089,191	10.92 %
PPO	-105.89	1,128,745	12.50 %
FFD	-120.6	1,207,832	13.86 %
Round Robin	-131.3	1,096,902	15.91 %
Agentic-RL	-137.66	1,211,862	16.70 %
DQN	-140.95	1,104,786	17.06 %

$\alpha = 1$, $\beta = 50$ · heterogenous fleet

WHAT THE NUMBERS SAY



SJF leads

10.92 % SLA AND lowest power (1.089 M). On clean Poisson with uniform job specs, a myopic length-aware heuristic dominates both axes — no learned agent beats it.



PPO is the strongest learner

12.50 % SLA — clearly ahead of every other learned agent and both heuristic fall-backs (RR, DQN), sitting just 1.6 pt behind SJF on SLA and second-best on reward.



DQN struggles at scale

Worst SLA (17.06 %), about 1.2 pt behind RR — the $K \cdot N + 1 = 251$ -way joint discrete action space is a known DQN scaling weakness, not a true policy collapse.



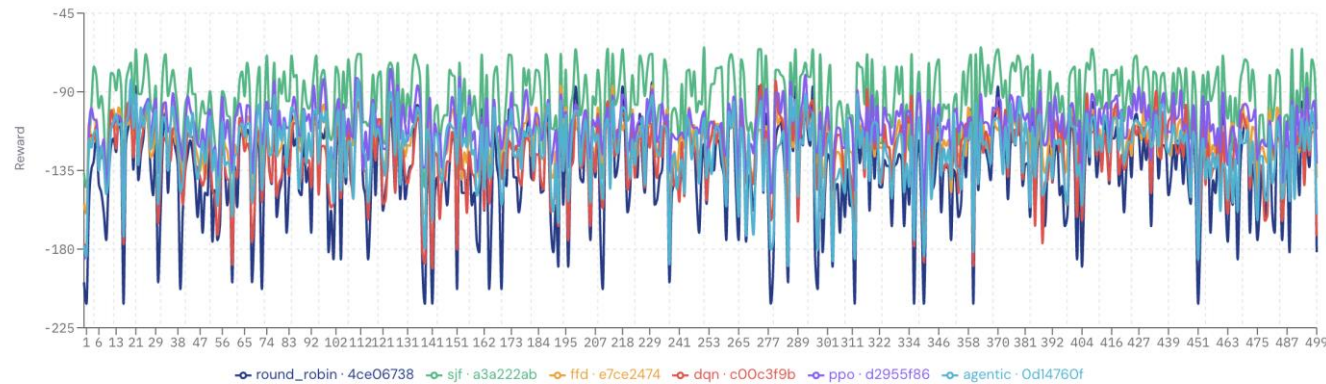
Agentic-RL trails

16.70 % SLA — second-worst. The Bernoulli supervisor flips between sub-agents step-to-step, inducing high variance and preventing either critic from converging on a coherent policy.

Poisson workload · N=50 het. fleet · 500 episodes

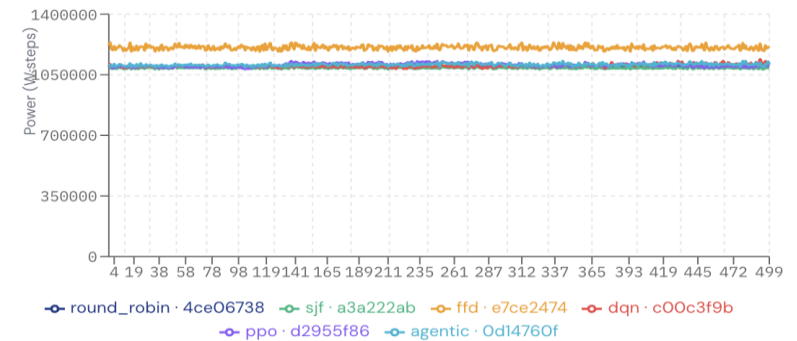
Training Curves

Dashed line = optimal reward ceiling (0.00)



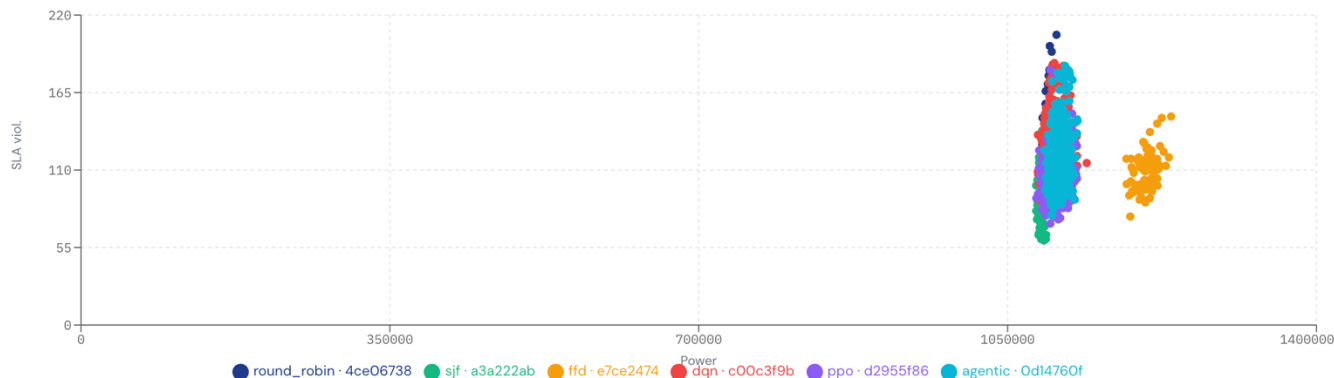
Power per episode

One line per selected run



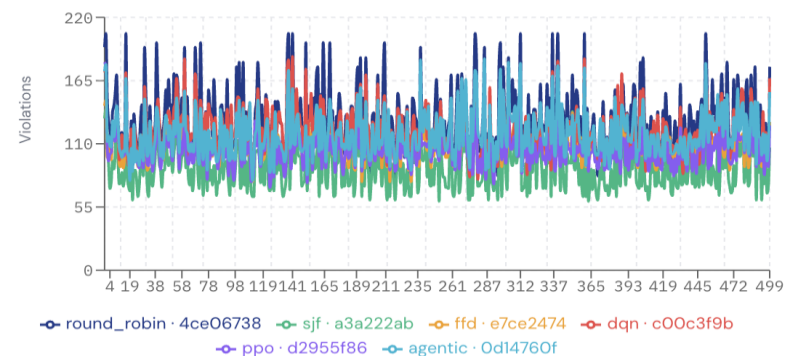
Power vs. SLA (per episode)

Each point = one training episode



SLA violations per episode

One line per selected run



Real-trace job-spec distribution + Poisson timing



Held-out test pool · same seeds & fleet as synthetic

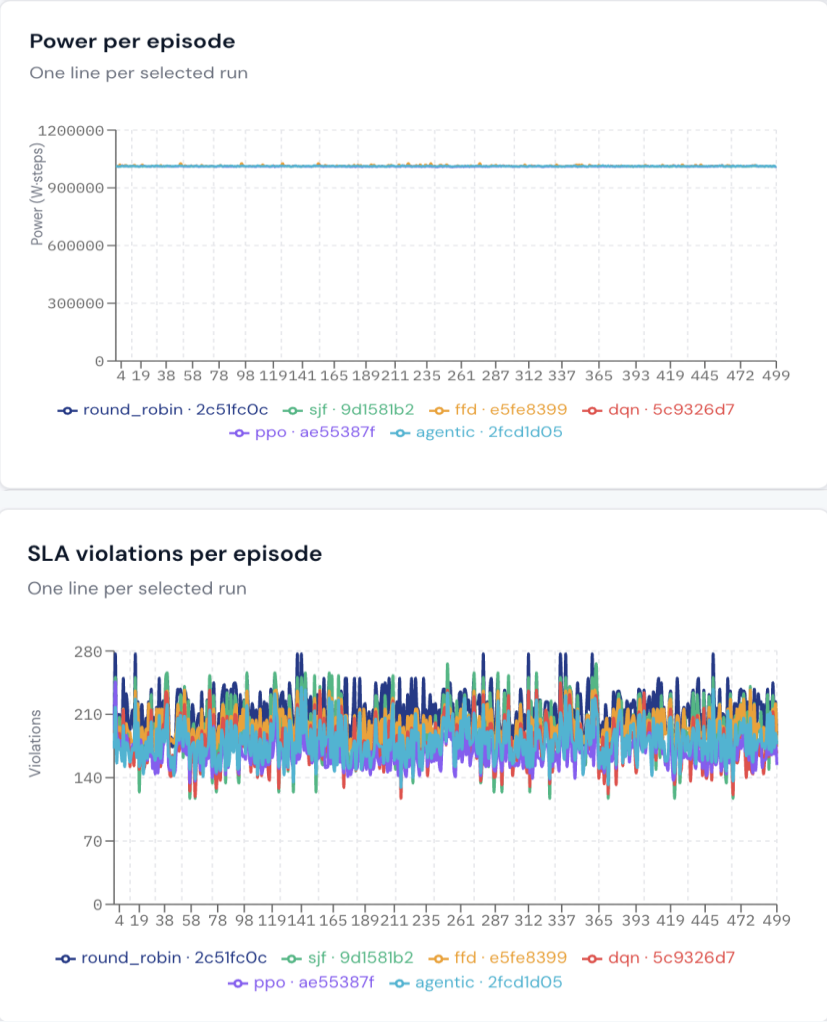
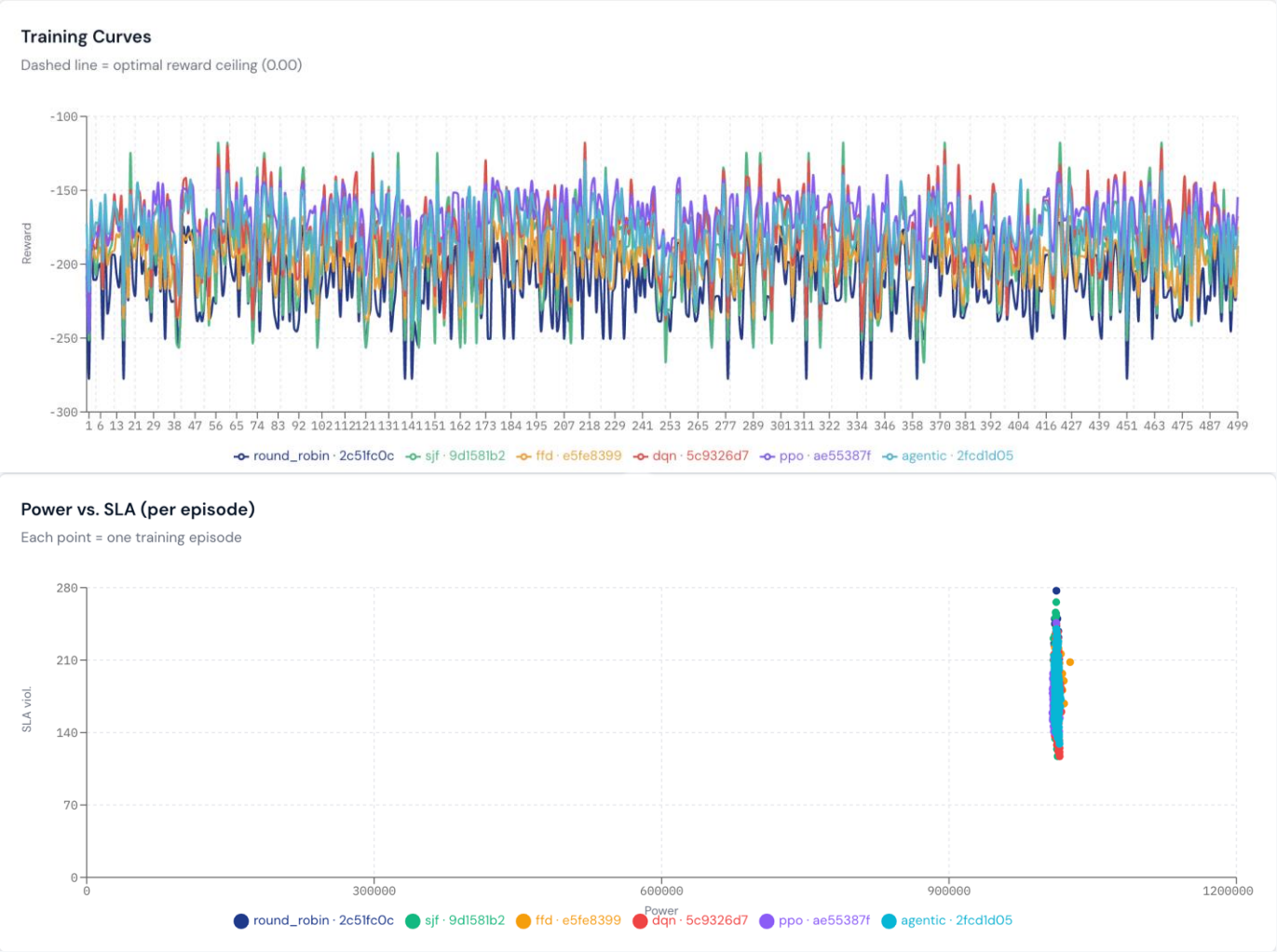
Agent	Mean R	Mean Power	SLA rate
DQN	-180.3	1,013,201	22.51 %
Agentic-RL	-181.6	1,013,322	22.67 %
SJF	-179.8	1,011,209	22.47 %
PPO	-164.1	1,012,233	20.49 %
FFD	-194.3	1,012,712	24.26 %
Round Robin	-210.6	1,012,195	26.31 %

CPU/mem/duration sampled from Google v2 trace, Poisson timing

CROSS-WORKLOAD ANALYSIS

- PPO takes the lead under distribution shift**
Best 20.49 % (PPO) vs. worst 26.31 % (RR) — a 5.8-pt spread, comparable to the 6.1-pt spread on synthetic, but the ordering flips: PPO is now ahead of every heuristic, not chasing them.
- PPO leads, the rest cluster**
PPO at 20.49 % is roughly 2 pt ahead of SJF (22.47 %); DQN and Agentic-RL fall within 0.2 pt of SJF, statistically tied. Realistic job specs let a learned policy exploit timing structure that heuristics cannot see.
- RR is the laggard**
26.31 % SLA — RR's blind cycling cannot cope with the heavier-tailed job durations in real-trace specs.
- Power is essentially flat**
All agents within ± 0.2 % of each other on mean power — the problem is SLA-bound under this workload, not power-bound. The interesting differentiation lives on the SLA axis, where PPO's 2-pt lead is the headline.

Real-trace job-spec distribution + Poisson timing



Poisson workload | Sleep/Wake server Actions


 Held-out test pool · same seeds & fleet as synthetic

Agent	Mean R	Mean Power	SLA rate
DQN	-1191.61	997,579	26.67 %
Agentic-RL	-1372.01	1,041,943	33.95 %
SJF	-1111.84	1.647,891	4.08 %
PPO	-1128.24	1,584,518	4.03 %
FFD	-1111.69	1,659,447	3.25 %
Round Robin	-1081.31	1,643,187	4.03 %

CPU/mem/duration sampled from Google v2 trace, Poisson timing

SLEEP / WAKE ANALYSIS

- PPO is the only balanced learner**

PPO matches heuristic SLA (4.03 %) — tied with RR and within 1 pt of FFD's 3.25 % — while pulling mean power down to 1.58 M, ~4–5 % below every heuristic. The only agent that uses sleep without breaking the SLA budget.
- DQN & Agentic-RL over-sleep**

Both slam servers asleep too aggressively. DQN drops mean power to 997 k (~40 % below heuristics) but lets SLA climb to 26.67 %; Agentic-RL is worse on both axes than DQN — 1.04 M power and 33.95 % SLA — the supervisor's flip-flopping prevents either sub-policy from settling.
- Heuristics cluster, can't touch power**

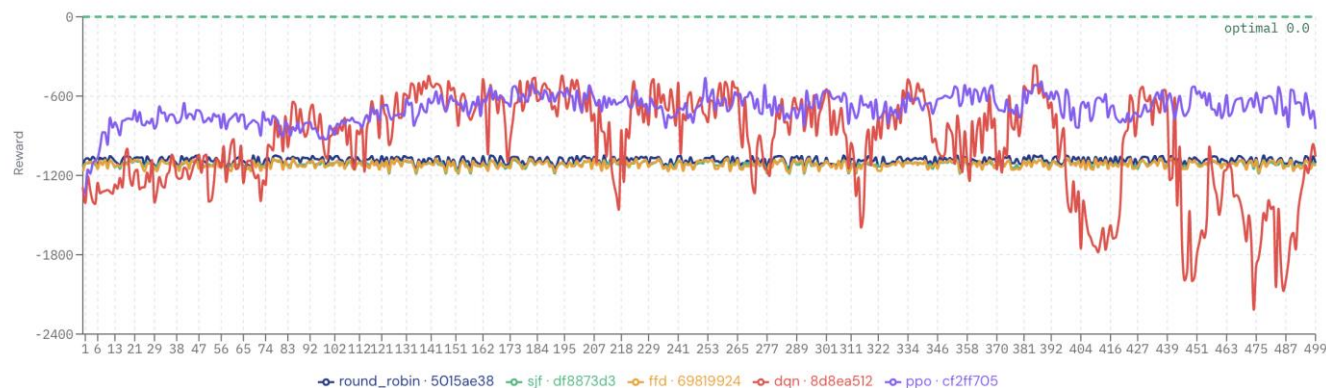
FFD / SJF / RR all sit around 1.64–1.66 M power and 3.25–4.08 % SLA — a tight 4-way tie. Without sleep/wake actions in their playbook, they have no lever to move power; SLA is the only axis they differentiate on.
- Sleep/wake unlocks a real power axis**

Power spread is now 660 k (~40 %) — DQN 997 k vs FFD 1.66 M — versus the ±0.5 % flat band on the always-on baseline. The headline isn't who wins outright; it's that the agents finally have a power lever, and PPO is the only one that learned to pull it without wrecking SLA.

Poisson workload | Sleep/Wake server Actions

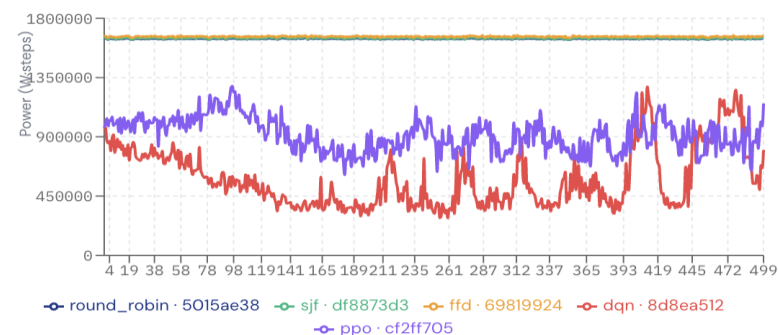
Training Curves

Dashed line = optimal reward ceiling (0.00)



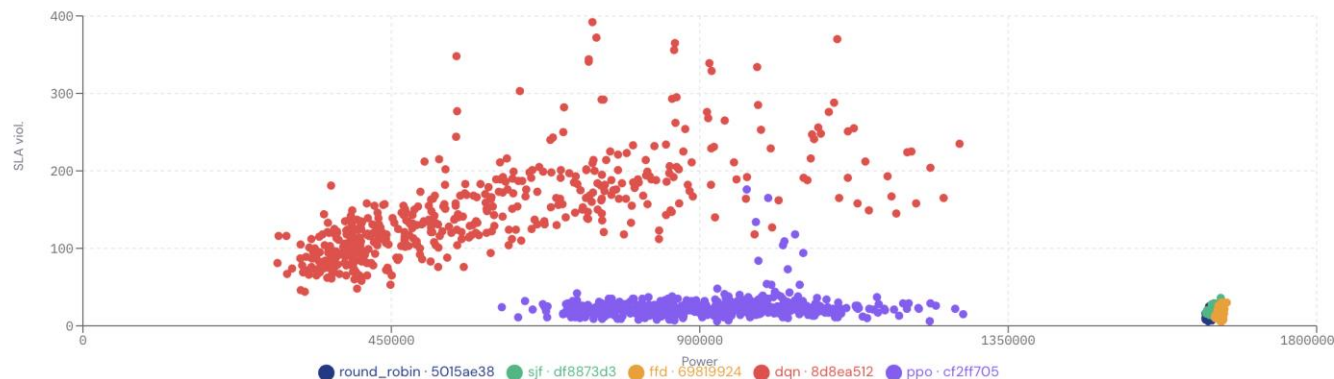
Power per episode

One line per selected run



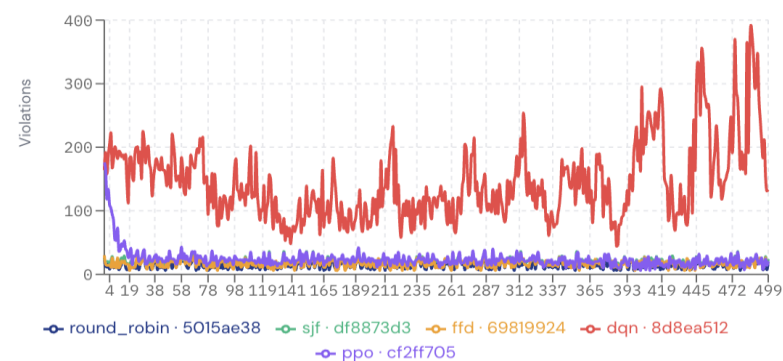
Power vs. SLA (per episode)

Each point = one training episode



SLA violations per episode

One line per selected run



Real-trace job-spec distribution + Sleep/Wake server action



Held-out test pool · same seeds & fleet as synthetic

Agent	Mean R	Mean Power	SLA rate
DQN	-372.60	256,996	10.68 %
Agentic-RL	-1383.17	979,801	36.52 %
SJF	-1232.34	1,604,132	11.65 %
PPO	-1257.29	1,572,045	11.30 %
FFD	-1257.06	1,609,294	12.55 %
Round Robin	-1259.57	1,605,769	12.90 %

CPU/mem/duration sampled from Google v2 trace, Poisson timing

SLEEP / WAKE · GOOGLE v2 ANALYSIS



DQN dominates both axes

Best SLA (10.68 %) AND lowest power (257 k, ~84 % below the heuristic cluster) — DQN's value-iteration head exploits the Google-v2 timing structure, putting servers to sleep aggressively without missing deadlines. Reward (-372) is in a different league.



PPO is the steady balanced learner

11.30 % SLA — best among the heuristic-level-power agents and ~0.4–1.6 pt ahead of every heuristic. Mean power 1.572 M, ~2 % below the heuristic cluster. Confirms PPO's synthetic result: balanced gains, no SLA sacrifice.



Heuristics cluster, no power lever

SJF / FFD / RR all sit at 1.60–1.61 M power with 11.65–12.90 % SLA — a tight 3-way tie. Without sleep/wake actions they cannot move power, and SLA varies only weakly with dispatch order under real-trace specs.



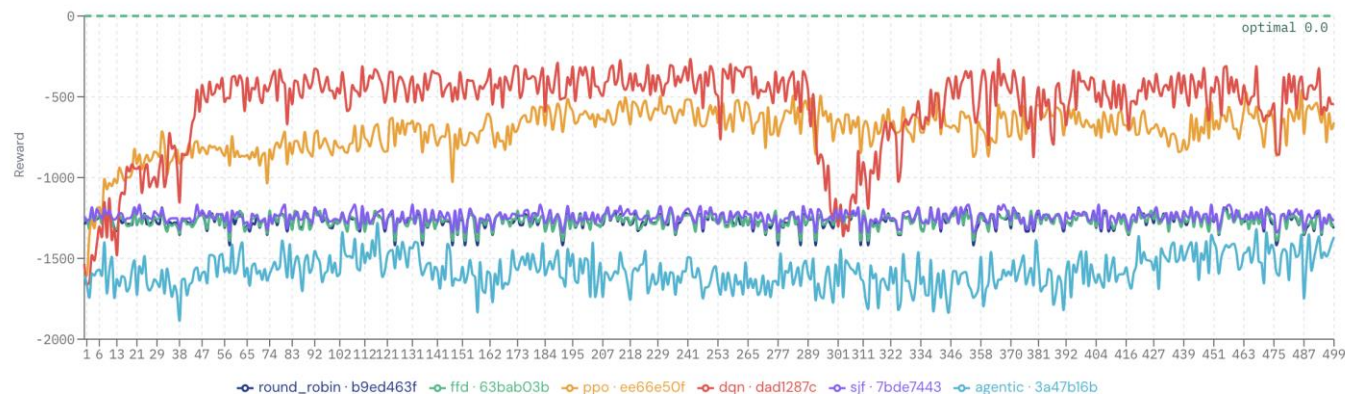
Agentic-RL collapses on SLA

36.52 % SLA — far worse than any other agent — even though power (980 k) is in DQN's neighbourhood. The supervisor's Bernoulli flip between sub-agents prevents the SLA critic from holding the deadline; flagged for redesign in Future Work.

Real-trace job-spec distribution + Sleep/Wake server action

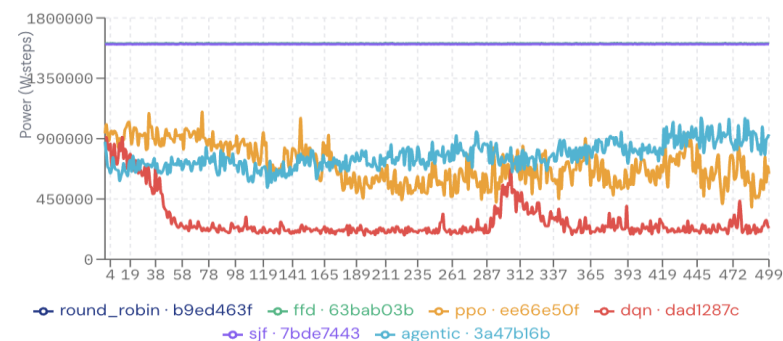
Training Curves

Dashed line = optimal reward ceiling (0.00)



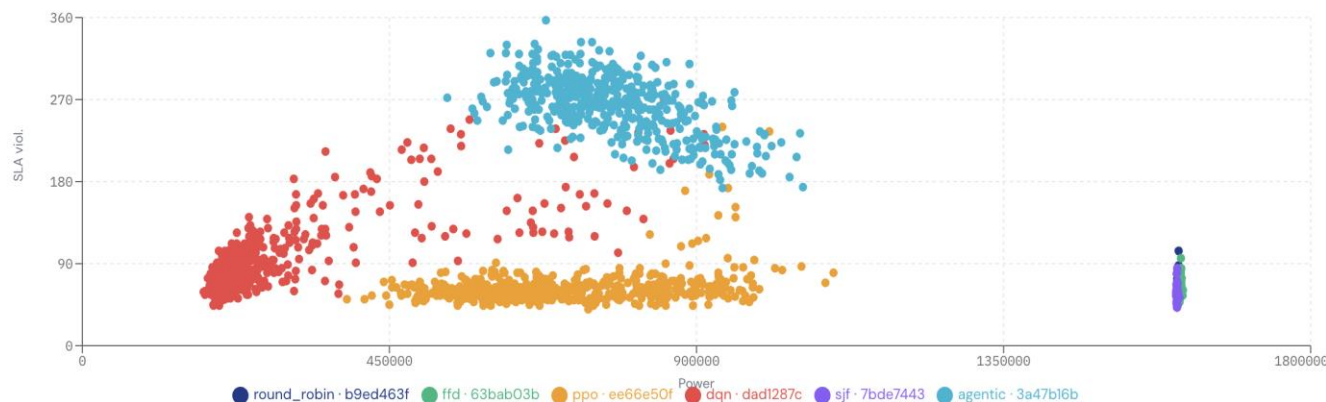
Power per episode

One line per selected run



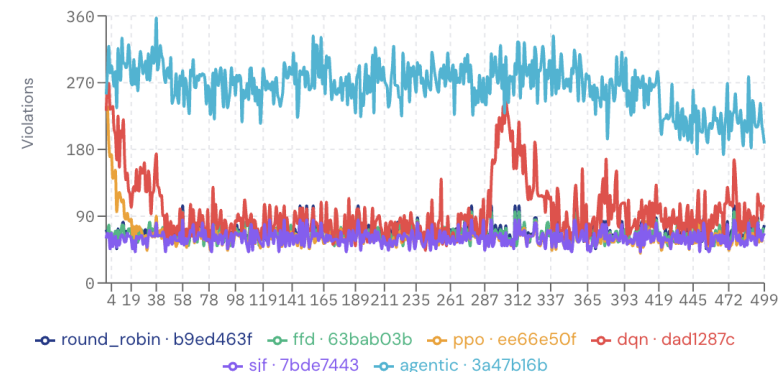
Power vs. SLA (per episode)

Each point = one training episode



SLA violations per episode

One line per selected run



Approach 2 — what offline-RL can and cannot guarantee

WHAT WE BUILT

- Lagrangian CMDP solver: Q_r (reward) + Q_c (SLA cost) trained jointly via Fitted Q Iteration
- Policy: $\pi = \arg\max (Q_r - \lambda \cdot Q_c)$, with λ updated by online dual ascent
- Conservative Q Learning-style conservative penalty keeps Q_c honest on out-of-distribution actions
- Trains on logged transitions from a tunable RR / SJF / FFD + uniform mix
- Dedicated React page: live FQI loss, λ trajectory, slack diagnostic — one-click reproducible

WHAT WE LEARNED

- Vanilla FQI was over-optimistic: Q_c predicted ~ 0 cost on unseen actions
- Without CQL the policy walked off-distribution, hitting 13 % SLA while believing it was safe
- Adding the CQL penalty fixed it cleanly — policy stayed in-distribution and tracked the dataset
- Result: a stable, well-behaved offline learner that is bounded by data

WHAT IT MEANS

- Offline CMDP is the right tool when live exploration is unsafe — and it works as advertised
- The CQL pessimism on Q_c saturated the dual variable.
- The behavior policies don't explore sleep/wake yet
- Tighter SLA targets just need a stronger demonstrator OR an online dual-update extension (PPO-Lagrangian)

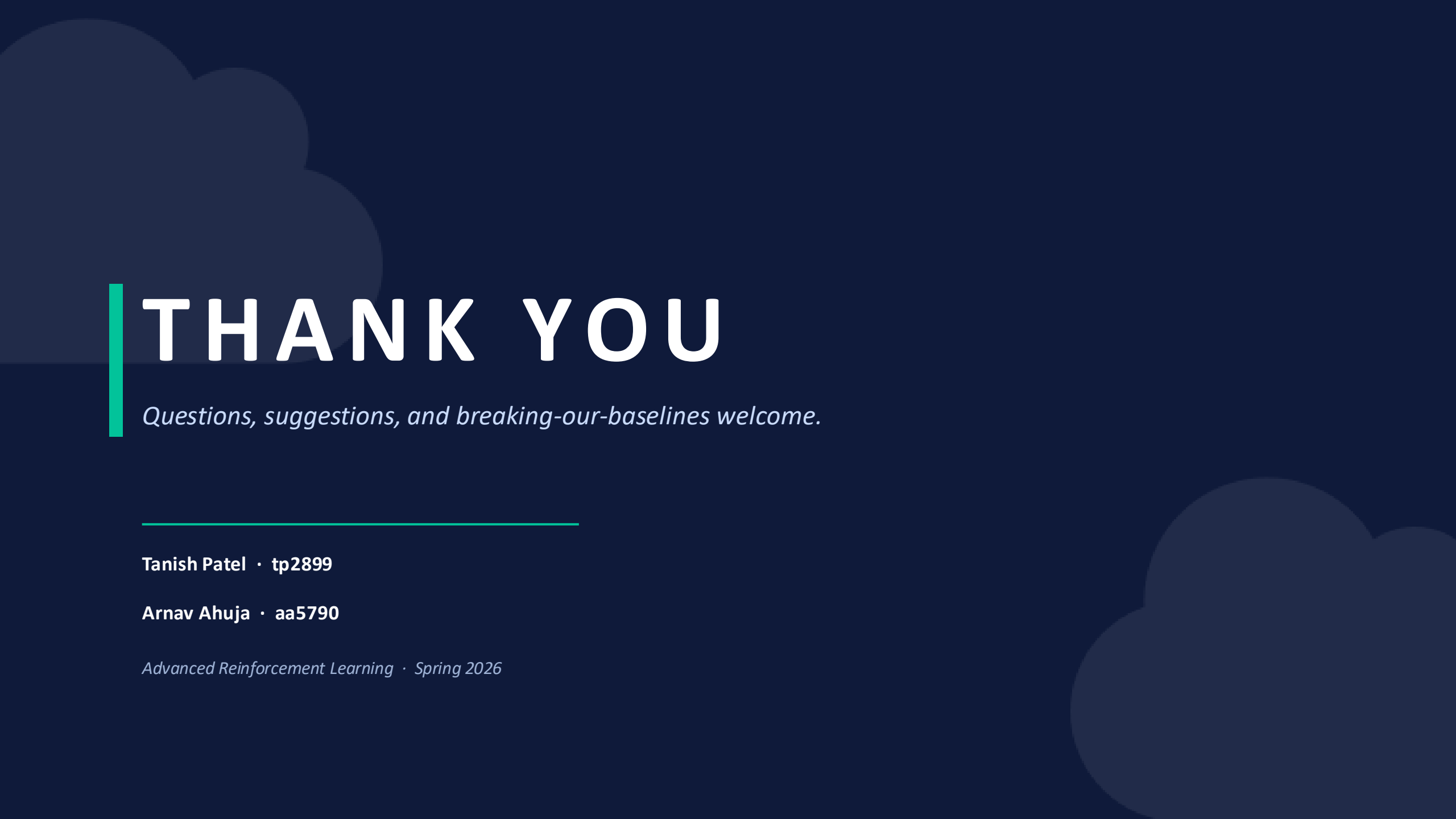
Headline takeaways and what we'd build next

HEADLINE TAKEAWAYS

- 1 Both proposed approaches (Agentic + Offline-CMDP) shipped end-to-end through one FastAPI + React UI.
- 2 On clean Poisson, SJF beats every learned agent on both SLA and power — PPO closes the gap (12.5 % vs 10.9 % SLA) but doesn't pass it.
- 3 On Google-v2-sampled the band tightens to 4 pts; DQN, Agentic-RL and PPO become statistically tied with SJF — distribution shift erases the heuristic edge.
- 4 Offline CMDP hits the demonstrator-ceiling wall: Lagrangian relaxation alone cannot deliver hard SLA guarantees.

WHAT WE'D BUILD NEXT

1  Action shielding Hard runtime override: when the empirical violation budget depletes, the wrapper takes over — guaranteed per-episode SLA cap, regardless of the policy.	2  PPO-Lagrangian (online) Move the dual variable into the live training loop, so λ reacts to real env violations instead of frozen logged costs — soft-constraint version of CMDP, end-to-end.	3  CPO with safety critic Constrained Policy Optimisation with hard rejection via a calibrated safety critic — formal cost bounds at each policy update.	4  Non-stationary loads Replace Poisson timing with the real Google-v2
---	--	--	--



THANK YOU

Questions, suggestions, and breaking-our-baselines welcome.

Tanish Patel · tp2899

Arnav Ahuja · aa5790

Advanced Reinforcement Learning · Spring 2026